

Task Management Platform - Presentation Technique Complete

Version basee sur l'etat du code

Date: 2026-03-09

1. Objectif de l'application

Task Management Platform est une plateforme microservices pour gerer des utilisateurs et des taches, avec:

- decouplage inter-services par messaging Kafka
- securisation des appels API par JWT
- service discovery via Eureka
- configuration centralisee via Spring Cloud Config

Finalite metier:

- creer des utilisateurs
- exposer des APIs de gestion de taches
- propager des evenements de domaine (ex: `user.created`)
- permettre l'evolution vers une architecture event-driven robuste

2. Vue d'ensemble architecture

Architecture logique:

- Frontend React (`frontend/`), proxy vers API Gateway
- API Gateway (`infrastructure/api-gateway`) pour routage + filtrage auth
- Discovery Service (`infrastructure/discovery-service`) avec Eureka Server
- Config Service (`infrastructure/config-service`) pour distribuer la config
- User Service (`services/user-service`) en architecture hexagonale partielle
- Task Service (`services/task-service`) avec endpoint REST et consumer Kafka
- Kafka + Zookeeper (`docker-compose.kafka.yml`) pour event bus
- PostgreSQL (`docker-compose.db.yml`) pour persistance user + outbox

Flux principal:

1. Client appelle API Gateway
2. Gateway route vers service cible via `lb://...` + Eureka
3. User Service cree un user et ecrit un event dans outbox
4. Scheduler outbox publie vers Kafka topic `user.created`
5. Task Service consomme `user.created`

3. Structure Maven et modules

Racine Maven:

- Java 21, Spring Boot 3.4.1, Spring Cloud 2024.0.1
- Modules: `shared`, `infrastructure`, `services`

Infrastructure:

- `config-service`
- `discovery-service`
- `api-gateway`

Services:

- **user-service:** `user-api`, `user-application`, `user-domain`, `user-infrastructure`
- **task-service:** `task-api`, `task-application`, `task-domain`, `task-infrastructure`

Note de coherence dependances:

- **user-service/pom.xml** redefinit Spring Boot 3.3.5 et Spring Cloud 2023.0.4 localement.
- Le parent racine est en 3.4.1 / 2024.0.1.
- Recommandation: aligner toutes les versions pour eviter des incompatibilites runtime.

4. Detail des dependances critiques

4.1 API Gateway

Dependances clefs:

- `spring-cloud-starter-gateway`
- `spring-cloud-starter-netflix-eureka-client`
- `spring-boot-starter-security`
- `io.jsonwebtoken:jjwt-api/impl/jackson` (0.12.5)

Observation:

- `spring-boot-starter-security` est declare 2 fois dans le POM.
- Cela doit etre nettoye.

4.2 User Service (module user-api)

Dependances clefs:

- REST: `spring-boot-starter-web`
- Discovery: `spring-cloud-starter-netflix-eureka-client`
- Observabilite: `spring-boot-starter-actuator`
- Messaging: `spring-kafka`
- Security token: `jjwt` 0.12.6
- Persistence JDBC: `spring-boot-starter-jdbc`, `postgresql`

4.3 Task Service (module task-api)

Dependances clefs:

- REST: `spring-boot-starter-web`
- Discovery: `spring-cloud-starter-netflix-eureka-client`
- Messaging: `spring-kafka`
- Modules internes: `application/domain/infrastructure`

5. JWT - Implementation actuelle et fonctionnement

5.1 Generation du token

user-service expose POST `/auth/login`:

- classe: `AuthController`
- service: `JwtService`
- token signe HMAC avec secret `jwt.secret`
- claims:
 - `sub` = `userId`
 - `iat`
 - `exp` (1h)
 - `roles`: `["USER"]`

5.2 Verification au Gateway

JwtWebFilter (gateway):

- passe sans auth pour `/auth/**` et `/actuator/**`
- exige header `Authorization: Bearer ...` pour le reste
- validation actuelle: compare la valeur a `demo-token`

Conclusion importante:

- Le JWT est genere correctement cote user-service
- Le gateway ne valide pas encore cryptographiquement ce JWT
- Etat actuel = protection de demonstration, pas securite production

5.3 Ce que le JWT sert ici

- authentifier les requetes entre client et APIs
- centraliser le controle d'accès a l'entree (gateway)
- preparer l'ajout futur de roles/scopes et ACL par endpoint

6. Kafka - Implementation actuelle et fonctionnement

6.1 Infrastructure Kafka

`docker-compose.kafka.yml` lance:

- Zookeeper (2181)
- Kafka broker (9092 host, 29092 interne docker)

Ce compose est necessaire mais insuffisant seul:

- il fournit le bus technique
- il ne fournit pas la logique metier producer/consumer

6.2 Production d'évenements (user-service)

Pattern implemente:

- Use case `CreateUserUseCase` cree user + cree `OutboxEvent` JSON
- `JdbcOutboxRepository` persiste en table `outbox_events`
- `OutboxScheduler` lit les events `PENDING` et publie Kafka
- topic publie: `user.created`
- puis `markSent` ou `markFailed` (retry exponentiel simple)

Interet:

- resilience en cas de panne Kafka temporaire
- pas de perte immediate d'évenements
- pattern outbox transactionnel (base de depart solide)

6.3 Consommation d'évenements (task-service)

Consumer present:

- classe: `UserCreatedEventListener`
- `@KafkaListener(topics="${kafka.topics.userCreated:user.created}")`
- deserialisation JSON via Jackson vers `UserCreatedEvent`
- action actuelle: log de reception (hook d'integration)

6.4 Topics et config

Config repository:

- `config-repo/user-service.yml`: `kafka.topics.userCreated: user.created`

- `config-repo/task-service.yml`: `userCreated`, `taskCreated`, `taskCompleted`

Local fallback task-service:

- `services/task-service/task-api/src/main/resources/application.yml`

7. Base de donnees et schema

SQL init:

- `docker/init/01_user_service.sql`:
 - `table users(id, email, created_at)`
- `docker/init/02-user-outbox.sql`:
 - `table outbox_events(...)`
 - `index status/next_attempt_at`
 - `index created_at`

Role du schema:

- **users**: source metier utilisateur
- **outbox_events**: file d'attente fiable avant publication Kafka

8. Service Discovery et Config centralisee

8.1 Discovery

`discovery-service`:

- Eureka Server sur port 8761
- clients (`gateway`, `user`, `task`, `config`) se registrent dessus

8.2 Config Service

`config-service`:

- port 8888
- mode `native`
- lit les fichiers depuis `file:./config-repo`

Attention execution:

- pour que `file:./config-repo` fonctionne, demarrer depuis la racine projet ou adapter le chemin absolu.

9. Endpoints principaux

- `POST /auth/login` (`user-service`) -> retourne un token
- `POST /users?email=...` (`user-service`) -> cree user + outbox
- `POST /kafka/user-created` (`user-service`, test) -> publie event test
- `GET /tasks` (`task-service`) -> retourne une liste stub

Via gateway:

- `/users/**` route vers `lb://USER-SERVICE`
- `/tasks/**` route vers `lb://TASK-SERVICE`

10. Comment realiser l'application (guide pas a pas)

10.1 Prerequis

- JDK 21
- Maven 3.9+

- Docker Desktop

10.2 Lancer l'infra externe

```
docker compose -f docker-compose.db.yml up -d
docker compose -f docker-compose.kafka.yml up -d
```

10.3 Lancer les services (ordre recommande)

1. discovery-service (8761)
2. config-service (8888)
3. user-service (8081)
4. task-service (8083)
5. api-gateway (8082)

Exemples Maven:

```
mvn -pl infrastructure/discovery-service spring-boot:run
mvn -pl infrastructure/config-service spring-boot:run
mvn -pl services/user-service/user-api spring-boot:run
mvn -pl services/task-service/task-api spring-boot:run
mvn -pl infrastructure/api-gateway spring-boot:run
```

10.4 Tester le flux complet

1. Generer un token:

```
curl -X POST http://localhost:8081/auth/login
```

2. Creer un user:

```
curl -X POST "http://localhost:8081/users?email=test@example.com"
```

3. Observer:

- ligne inseree dans `users`
- ligne outbox PENDING puis SENT
- reception de `user.created` dans logs `task-service`

11. Comment ca marche techniquement

11.1 Chemin synchrone API

Client -> Gateway -> Service cible -> reponse HTTP

11.2 Chemin asynchrone evenementiel

CreateUserUseCase -> outbox_events -> OutboxScheduler -> Kafka topic -> Task consumer

Separation des responsabilites:

- API synchrone pour UX immediat
- event asynchrone pour propagation inter-domaines

12. Pourquoi cette architecture est utile

- Scalabilite: services deployables independamment
- Decouplage: Kafka evite les appels directs systematiques
- Resilience: outbox limite la perte d'evenements
- Gouvernance: gateway centralise politiques d'accès
- Evolutivite: ajout d'autres consumers sans toucher user-service

13. Limites actuelles et plan d'amélioration

Priorité haute:

- remplacer validation `demo-token` du gateway par vraie validation JWT
- harmoniser versions Spring Boot/Cloud entre parent et user-service
- nettoyer duplication dependency security dans gateway

Priorité moyenne:

- remplacer topic hardcode `"user.created"` par `KafkaTopicsProperties` dans `producer/scheduler`
- brancher consumer task-service sur logique métier (creation tache auto, projection, etc.)
- ajouter DLQ/retry policy Kafka centralisée

Priorité qualité:

- tests d'intégration Testcontainers (Postgres + Kafka)
- tracing distribue (OpenTelemetry)
- dashboards Actuator/Prometheus/Grafana

14. Resume executif (1 slide)

- L'application implemente deja la base d'une architecture microservices event-driven.
- Kafka est reellement utilise (producer + outbox + consumer), pas seulement docker.
- JWT existe cote emission mais validation gateway est encore en mode demo.
- Le socle est bon pour un POC avance; 3 correctifs permettent de passer vers un niveau pre-prod.